

quartz-layout-box

Dokumentation · Version 0.2.0 · Stand 3. September 2026 · Quartz 5 Community-Plugin

1. Was das Plugin macht
2. Funktionsweise
3. Installation und Einbindung
4. Optionen
5. Platzhalter
6. Steuerung per Frontmatter
7. Markdown-Snippets
8. Titel und aufklappbare Box
9. Markup und Styling
10. Verhalten bei Fehlern
11. Mehrere Instanzen
12. Umstieg von Version 0.1
13. Entwicklung

1. Was das Plugin macht

Die Layout-Box fügt an einer beliebigen Stelle des Quartz-Layouts ein Stück HTML ein: in der linken oder rechten Seitenleiste, im Header, im Footer oder vor beziehungsweise nach dem Seiteninhalt. Typische Einsatzfälle sind ein Titelblock mit Logo, ein Untertitel, ein Hinweiskasten, ein Spenden- oder Newsletter-Link oder eine kurze Beschreibung der Site.

Der Inhalt kommt aus einer Snippet-Datei im Ordner `quartz/static/snippets/` der Site oder direkt aus der `quartz.config.yaml`. Snippets können HTML oder Markdown sein. Beim Rendern werden Platzhalter wie `{{siteTitle}}` ersetzt. Das Plugin arbeitet vollständig zur Build-Zeit und liefert kein JavaScript an den Browser aus.

2. Funktionsweise

Quartz lädt das Plugin als reine Komponente. Für jeden Eintrag in der `quartz.config.yaml`, der einen `layout`-Block besitzt, wird eine eigene Instanz mit den dort angegebenen Optionen erzeugt. Beim Bau jeder Seite läuft dann folgender Ablauf:

1. Frontmatter prüfen

Ist die Box für diese Seite ausgeblendet oder wurde ein anderes Snippet gewählt?

2. Quelle bestimmen

Inline-HTML aus der Konfiguration hat Vorrang, sonst die Datei `dir/file`.

3. Datei lesen

Der Pfad muss innerhalb von `dir` liegen. Gelesen wird mit Cache: nur wenn sich die Änderungszeit der Datei geändert hat, wird sie neu geladen.

4. Markdown rendern

Endet die Datei auf `.md`, wird sie zu HTML umgewandelt. Das Ergebnis landet ebenfalls im Cache.

5. Platzhalter ersetzen

Seiten- und Site-Werte werden eingesetzt, HTML-Sonderzeichen dabei maskiert.

6. Markup ausgeben

Ein `div` mit Klasse `layout-box`, optional mit Überschrift oder als aufklappbares `details`-Element.

Weil die Datei bei jedem Render gegen ihre Änderungszeit geprüft wird, greifen Änderungen am Snippet im Modus `npx quartz build --serve` sofort, ohne Neustart. Im normalen Build kostet das pro Seite nur einen Dateisystem-Stat.

3. Installation und Einbindung

```
npx quartz plugin add github:boxi-os/quartz-layout-box
```

Danach ein Snippet anlegen, zum Beispiel `quartz/static/snippets/snippet.html`, und das Plugin in der `quartz.config.yaml` eintragen. Der `layout`-Block bestimmt, wo die Box erscheint:

```
plugins:
  - source: github:boxi-os/quartz-layout-box
    enabled: true
    options:
      file: snippet.html
    layout:
      position: left          # left | right | header | footer | beforeBody | afterBody
      priority: 15           # Reihenfolge innerhalb der Position, kleiner = weiter oben
      display: desktop-only  # all | desktop-only | mobile-only
```

Ohne `options` gelten die Standardwerte: Datei `snippet.html` im Ordner `quartz/static/snippets`.

4. Optionen

Option	Typ	Standard	Beschreibung
<code>file</code>	string	<code>snippet.html</code>	Dateiname des Snippets innerhalb von <code>dir</code> . Dateien mit Endung <code>.md</code> werden als Markdown gerendert. Wird ignoriert, wenn <code>html</code> gesetzt ist.
<code>dir</code>	string	<code>quartz/static/snippets</code>	Ordner für Snippet-Dateien, relativ zur Wurzel der Quartz-Site. Pfade, die aus diesem Ordner hinausführen, werden abgelehnt.
<code>html</code>	string	–	HTML direkt in der Konfiguration. Hat Vorrang vor <code>file</code> . Praktisch für Einzeiler.
<code>className</code>	string	–	Zusätzliche CSS-Klasse(n) neben der festen Klasse <code>layout-box</code> . Damit lassen sich mehrere Instanzen getrennt stylen.
<code>title</code>	string	–	Überschrift über dem Inhalt, gerendert als <code>h3</code> wie bei anderen Sidebar-Komponenten.
<code>collapsible</code>	boolean	<code>false</code>	Box als aufklappbares <code>details</code> -Element rendern. <code>title</code> wird zur Zeile, auf die man klickt. Ohne <code>title</code> wird die Option mit einer Warnung ignoriert.
<code>collapsed</code>	boolean	<code>false</code>	Box beim Laden zugeklappt anzeigen. Nur zusammen mit <code>collapsible</code> .
<code>placeholders</code>	boolean	<code>true</code>	Platzhalter der Form <code>{{name}}</code> im Snippet ersetzen. Mit <code>false</code> bleibt der Text unverändert.

Option	Typ	Standard	Beschreibung
frontmatterKey	string	layoutBox	Name des Frontmatter-Feldes, über das eine einzelne Seite die Box steuern kann (siehe Abschnitt 6).

Beispiel mit Inline-HTML, Titel und Aufklappen:

```
options:
  title: Hinweis
  collapsible: true
  collapsed: true
  html: |
    <p>Diese Site befindet sich im Aufbau.</p>
```

5. Platzhalter

Platzhalter werden auf jeder Seite neu ersetzt, die Werte werden HTML-maskiert. Unbekannte Platzhalter bleiben unverändert stehen. Leerzeichen innerhalb der Klammern sind erlaubt.

Platzhalter	Wert
{{title}}	Titel der aktuellen Seite aus dem Frontmatter
{{slug}}	Slug der aktuellen Seite, zum Beispiel notizen/meine-seite
{{root}}	Relativer Pfad zur Wurzel der Site (. auf der Startseite, ../.. zwei Ebenen tiefer). Funktioniert auch, wenn die Site unter einem Unterpfad liegt.
{{siteTitle}}	configuration.pageTitle aus der Quartz-Konfiguration
{{baseUrl}}	configuration.baseUrl, kann leer sein
{{locale}}	configuration.locale, zum Beispiel de-DE
{{frontmatter.<feld>}}	Beliebiges Frontmatter-Feld der Seite. Listen werden mit Komma verbunden, Objekte werden nicht ersetzt.

Empfehlung: Links und Bildpfade im Snippet mit {{root}} statt mit absolutem / schreiben.

```
<h2><a href="{{root}}/">{{siteTitle}}</a></h2>
<p>Politische Notizen aus der Provinz</p>


```

6. Steuerung per Frontmatter

Einzelne Seiten können die Box über ein Frontmatter-Feld beeinflussen. Der Feldname ist über frontmatterKey einstellbar, Standard ist layoutBox.

Frontmatter	Wirkung
<code>layoutBox: false</code>	Box auf dieser Seite ausblenden.
<code>layoutBox: impressum.html</code>	Anderes Snippet verwenden, relativ zu <code>dir</code> .
<code>layoutBox:</code> <code>html: "<p>Nur hier</p>"</code>	Inline-HTML nur für diese Seite. Alternativ <code>file:</code> für eine andere Datei oder <code>hidden: true</code> zum Ausblenden.

Bei mehreren Instanzen bekommt jede ihren eigenen `frontmatterKey`, zum Beispiel `layoutBox` und `layoutBoxMobile`, damit sie unabhängig voneinander gesteuert werden können.

7. Markdown-Snippets

Endet `file` auf `.md`, wird die Datei beim Build mit Standard-Markdown gerendert (GitHub Flavored Markdown, eingebettetes HTML erlaubt). Platzhalter funktionieren auch hier.

```
## {{siteTitle}}

Politische Notizen aus der *Provinz*.

- [Über diese Site]({{root}}/ueber)
- [Kontakt]({{root}}/kontakt)
```

Markdown-Snippets laufen nicht durch die Quartz-Inhaltspipeline. Wikilinks in doppelten eckigen Klammern, Callouts und andere Obsidian-Syntax werden dort nicht aufgelöst. Normale Markdown-Links funktionieren.

8. Titel und aufklappbare Box

Mit `title` erhält die Box eine Überschrift. Mit `collapsible: true` wird sie zu einem aufklappbaren Element, bei dem der Titel als Klickzeile dient. `collapsed: true` startet zugeklappt. Das funktioniert ohne JavaScript über das HTML-Element `details`; der Browser merkt sich den Zustand nicht über Seitenwechsel hinweg.

9. Markup und Styling

Das Plugin bringt nur minimale, farblose Regeln mit. Das erzeugte Markup:

```

<div class="layout-box [className]">
  <h3 class="layout-box-title">...</h3>          <!-- nur mit title -->
  <div class="layout-box-content">...Snippet...</div>
</div>

<!-- mit collapsible: -->
<details class="layout-box [className]" open>
  <summary class="layout-box-title">...</summary>
  <div class="layout-box-content">...Snippet...</div>
</details>

```

Gestaltet wird in der `quartz/styles/custom.scss` der Site. Beispiel:

```

.layout-box h2 { margin: 0; font-size: 1.9rem; }
.layout-box p { font-style: italic; color: var(--dark); }
.layout-box-mobile { text-align: center; }

```

Mitgelieferte Regeln: Bilder werden auf die Breite der Box begrenzt. Bilder mit Klasse `img-light` erscheinen nur im hellen, Bilder mit `img-dark` nur im dunklen Theme. Eine fehlende Snippet-Datei wird im Serve-Modus als gestrichelter Kasten dargestellt (Klasse `layout-box-missing`).

10. Verhalten bei Fehlern

Situation	Verhalten
Snippet-Datei fehlt	Einmalige Warnung in der Konsole mit vollständigem Pfad. Im normalen Build wird nichts ausgegeben. Bei <code>quartz build --serve</code> erscheint an der Stelle ein gestrichelter Kasten mit dem erwarteten Pfad.
Pfad führt aus <code>dir</code> hinaus (z. B. <code>../secret.html</code>)	Wird abgelehnt, Warnung, keine Ausgabe.
Snippet ist leer	Keine Ausgabe, auch kein leeres <code>div</code> .
<code>collapsible</code> ohne <code>title</code>	Warnung, Box wird normal ohne Aufklappen gerendert.

Snippets werden ungefiltert als HTML eingefügt. Nur Dateien verwenden, die man selbst kontrolliert.

11. Mehrere Instanzen

Das Plugin kann mehrfach eingebunden werden, etwa mit unterschiedlichen Snippets für Desktop und Mobil. Jede Instanz bekommt eigene Optionen und einen eigenen `layout` -Block:

```
plugins:
  - source: github:boxi-os/quartz-layout-box
    enabled: true
    options:
      file: snippet.html
    layout:
      position: left
      priority: 15
      display: desktop-only
  - source: github:boxi-os/quartz-layout-box
    enabled: true
    options:
      file: snippet-mobile.html
      className: layout-box-mobile
      frontmatterKey: layoutBoxMobile
    layout:
      position: left
      priority: 15
      display: mobile-only
```

12. Umstieg von Version 0.1

- Die Option `datei` heißt jetzt `file`. Bestehende `quartz.config.yaml`-Einträge entsprechend anpassen.
- Das Plugin ist jetzt als reine Komponente deklariert. Der Beispiel-Transformer der Vorlage, der vorher ungewollt auf allen Seiten lief, ist entfernt.
- Die Regeln für `img-light` und `img-dark` liefert das Plugin jetzt selbst mit. Eigene Regeln dazu in der `custom.scss` können entfallen.
- Nach dem Aktualisieren des Plugins in der Site neu installieren, damit Quartz das geänderte Manifest liest.

13. Entwicklung

```
npm install
npm run check  # Typprüfung, Lint, Formatierung, Tests
npm run build  # erzeugt dist/ (wird committed, Quartz installiert daraus)
```

Nach jeder Änderung unter `src/` muss `npm run build` laufen und das aktualisierte `dist/` mit `committed` werden. Die wichtigsten Dateien: `src/components/LayoutBox.tsx` (Komponente), `src/placeholders.ts` (Platzhalter), `src/markdown.ts` (Markdown-Rendering), `src/types.ts` (Optionen), `package.json` (Quartz-Manifest im Feld `quartz`).